

# pubScan v3: co-authorship network construction

How the server built the network before, and how changes A (algorithm) and C (page-cache warm-up) alter it. Applies to **pubscan/index.py**, function **get\_author\_network**, and **dbase/Dockerfile**. Measurements taken on the production host on 2026-10-09.

## 1. Data the algorithm works with

Two SQLite tables in **pubscan.db** (5.1 million authors, 28.7 million publications, 13 GB) are used. Both are read with point lookups on their primary key.

| Table        | Key            | Columns used                | Meaning                                                                      |
|--------------|----------------|-----------------------------|------------------------------------------------------------------------------|
| authors      | orcid (TEXT)   | author_name, pmids          | pmids is a comma-separated string of every PMID the author appears on        |
| publications | pmid (INTEGER) | authors_name, authors_orcid | comma-separated, position-aligned lists of the paper's authors with an ORCID |

Two truncations are applied everywhere and are unchanged by A:

- **author\_pmids(orcid)** returns an author's PMIDs sorted newest first and cut to the **300** most recent.
- The center author's list is cut again to the **200** most recent publications before their co-authors are collected.

Because of the 300 cap, an author's publication set is an approximation for anyone with more than 300 papers; this was true before and is still true after the change.

## 2. Algorithm before the change

Input: the search string. The last whitespace-separated token is taken as the center ORCID. Output: a JSON object with **nodes\_all** (at most max\_nodes = 100 authors) and **edges\_all** (at most max\_edges = 1000 co-authorship edges), streamed to the browser after a series of progress lines.

### Step 1: center publications

Look up the center in **authors**, parse pmids, keep the 200 most recent.

### Step 2: candidate nodes

For each of those 200 PMIDs, read the publication row and iterate its (orcid, name) pairs in author order. Every ORCID seen for the first time becomes a candidate node, so candidates are ordered by first appearance, newest papers first. For each new candidate, its own pmids list is loaded once (newest first, cap 300). Candidates with an empty list are dropped. The center itself is a candidate because it appears on its own papers. Typical sizes: 100 to 2,300 candidates.

### Step 3: all-pairs intersection (the expensive step)

```
for a1, a2 in combinations(authors, 2):          # every unordered pair of ALL candidates
    common = list(set(p1).intersection(p2))      # two sets rebuilt from string lists, per pair
    if common:
        edges_all.append(edge_rec(a1, a2, common))
        nodes_degree[a1] += n; nodes_degree[a2] += n          # computed, never used
        if center_id in (a1, a2):
            nodes_cdegree[other] = n                        # shared papers with the center
            nodes_cdegree[center_id] = max(n, nodes_cdegree.get(center_id, 0))
```

With N candidates this is  $N(N-1)/2$  pairs. For 2,164 candidates it is 2.34 million pairs, each converting two lists of up to 300 strings into sets. The only outputs of this loop that influence the final result are

`nodes_cdegree` and the edges between nodes that end up being kept. `nodes_degree` is dead code.

#### Step 4: node selection

Candidates are sorted by `nodes_cdegree`, descending. Python's sort is stable, so ties keep first-appearance order. The first 100 with a degree greater than zero are kept. Anyone who shares no paper with the center within the two truncation windows is excluded.

#### Step 5: edge filtering

Of all edges from step 3, only those whose both endpoints are among the 100 kept nodes survive. This discards the vast majority of the work done in step 3.

#### Step 6: edge capping

If more than 1000 edges remain: keep every edge touching the center, then the 100 heaviest remaining edges by shared-paper count, then a random sample of the rest until 1000 is reached. Unchanged by A.

#### Measured cost (warm page cache)

| Center author        | candidates | pairs     | step 3 | whole request |
|----------------------|------------|-----------|--------|---------------|
| Gregor Rot           | 115        | 6,555     | 0.02 s | 0.02 s        |
| Jernej Ule           | 1,075      | 569,778   | 1.8 s  | 1.8 s         |
| Bin Zhang            | 1,012      | 505,515   | 1.5 s  | 1.5 s         |
| Nenad Ban            | 2,141      | 2,280,180 | 10.6 s | 10.9 s        |
| Christian von Mering | 2,164      | 2,327,403 | 13.1 s | 13.5 s        |

Step 3 is 97% of the request time for prolific authors. It is pure CPU, holds the Python GIL, and runs inside a single `mod_wsgi` daemon process, so concurrent visitors queue behind it.

### 3. Change A: compute center degree first, pair only the kept nodes

Steps 1, 2, 4 and 6 are untouched. Step 3 is split into two cheap parts and step 5 disappears because edges are only ever created between kept nodes.

#### Step 3a: one intersection per candidate

```
author_sets = {a: set(author_pmid[a]) for a in authors}      # each set built once, not once per pair
nodes_cdegree = {}
center_set = author_sets.get(center_id, set())
for a in authors:
    if a == center_id:
        continue
    num_common = len(center_set & author_sets[a])
    if num_common > 0:
        nodes_cdegree[a] = num_common
        nodes_cdegree[center_id] = max(num_common, nodes_cdegree.get(center_id, 0))
```

#### Step 4: node selection

Identical code: stable sort by nodes\_cdegree descending over first-appearance order, keep the first 100 with degree > 0.

#### Step 3b: all pairs among the kept nodes only

```
kept_ids = [node["id"] for node in nodes_all]                # at most 100
for a1, a2 in combinations(kept_ids, 2):                    # at most 4,950 pairs
    common = author_sets[a1] & author_sets[a2]
    if len(common) > 0:
        edges_all.append(edge_rec(a1, a2, common))           # same record format as before
```

#### Step 6: edge capping

Identical code.

### Why the result is the same

The final output is fully determined by two quantities: the center degree of each candidate, and the shared PMIDs of each pair of kept nodes.

- **Center degrees.** In the old loop, nodes\_cdegree[x] is written only when the pair (center, x) is processed, and its value is |set(center) & set(x)|. The other  $N(N-1)/2 - (N-1)$  pairs never touch it. Step 3a computes exactly that value for every x, including the center's own value as the maximum. Same values, same dictionary.
- **Node selection.** Same input dictionary, same stable sort over the same insertion order, same cut. Same 100 nodes in the same order.
- **Edges.** Every pair of kept nodes was also processed by the old loop with the same two sets, giving the same shared PMIDs, count, width and labels. Every other pair the old loop processed involved at least one discarded node and was deleted in step 5. Skipping those pairs changes nothing.
- **Edge capping.** Receives the identical edge list. random.sample draws from the same pool; which edges it picks varies per request exactly as it did before.

Work done:  $N + 4,950$  intersections instead of  $N(N-1)/2$ , and  $N$  set constructions instead of  $N(N-1)$ . For  $N = 2,164$  that is about 7,100 intersections instead of 2.34 million.

## What was removed

- The nodes\_degree accumulation (two places): never read.
- The post-hoc edge filter of step 5: redundant, edges are now created only between kept nodes.
- Per-pair set(p1), set(p2) construction: sets are built once per author.
- Progress texts that still said 150 nodes and 2000 edges now use the configured max\_nodes and max\_edges.

## Measured cost after A (warm page cache, same authors, real server function via WSGI)

| Center author        | before | after A | speed-up            |
|----------------------|--------|---------|---------------------|
| Gregor Rot           | 0.02 s | 0.04 s  | none (already fast) |
| Jernej Ule           | 1.8 s  | 0.19 s  | 9x                  |
| Bin Zhang            | 1.5 s  | 0.17 s  | 9x                  |
| Nenad Ban            | 10.9 s | 0.45 s  | 24x                 |
| Christian von Mering | 13.5 s | 0.73 s  | 18x                 |

The remaining time for the largest cases is dominated by the per-candidate progress line streamed to the browser (one JSON line per candidate, over 2,000 lines) and by building a node record with a PMID string for every candidate. Both are pre-existing and were left as they are.

## 4. Change C: warm the OS page cache at container start

### What makes a request slow when the cache is cold

SQLite keeps no copy of the database in memory. Every lookup walks a B-tree and reads 4 KB pages from the file through the kernel. The database files live on a RAID0 of two spinning disks (sda, sdb). A random 4 KB read that is not in RAM costs a head seek, measured at roughly 6 to 15 ms. A request for a prolific author performs about 200 publication lookups plus one author lookup per candidate, up to 2,300 lookups in total, each landing on a different page scattered across the 13 GB file.

| Live request to the container | first request (cold) | second request (warm) |
|-------------------------------|----------------------|-----------------------|
| Nenad Ban                     | 22.2 s               | 0.46 s                |
| Christian von Mering          | 21.0 s               | 1.09 s                |

A single IN (...) query instead of 200 point lookups was tested and did not help (3.0 s versus 3.1 s cold for 200 publications): the cost is the seeks, not the number of SQL statements.

### What the page cache is

Linux keeps every file block it has read in otherwise unused RAM, the page cache, until that memory is needed for something else. A block that is in the page cache is served without touching the disk. The cache is per file and per block, so a request warms only the pages it touched; a different author is cold again unless its rows share pages with earlier requests.

### What the change does

```
# dbase/Dockerfile, inside the CMD, just before Apache starts:
(cat /var/www/site/db/*.db > /dev/null 2>&1 &); \
```

cat reads both database files sequentially from start to end and discards the bytes. The side effect is that every block passes through the page cache and stays there. Sequential reads on these disks run at about 330 MB/s, so the 13 GB pubscan.db plus the 0.8 GB names.db are in RAM after about 41 s. It runs in the

background, so Apache starts immediately; requests arriving during those 41 s are simply as slow as before. The host has 62 GB of RAM with about 5 GB used by processes, so the 14 GB fits.

## Limits

- It is a one-shot warm-up at container start. Other workloads on the host (the expressRNA containers, a database rebuild, large file copies) can evict the pages again over time; the next request then pays the seeks until the pages are re-read.
- If that happens in practice, the next steps are a cron job repeating the same cat every few hours, or `vmtouch -l` to lock the files in memory permanently.
- Shrinking the publications table makes the file faster to warm and less likely to be evicted. Done, see section 6: dropping the never-read `authors_all` column takes `pubscan.db` from 13.69 GB to 9.38 GB (-31%).

## Take effect

The Dockerfile change needs an image rebuild and container restart: `cd dbase; ./build.sh; ./run.sh`. Note that `run.sh` deletes `logs/` (including `search.log`) before starting. Change A needs nothing: `index.py` is mounted into the container and `mod_wsgi` restarts its daemon when the script file changes. Full instructions are in `dbase/README.md`.

## 5. How equivalence was verified

A faithful re-implementation of the old steps 1 to 5 (all-pairs loop included) was run side by side with the patched `get_author_network`, called through a WSGI environment stub with `max_edges` raised so that step 6 does no random sampling. For each test author the following were compared: the list of kept node IDs in order, the set of edges as unordered (from, to) pairs, and the set of shared PMIDs carried by each edge.

| Center author        | kept nodes | edges | nodes identical             | edges identical |
|----------------------|------------|-------|-----------------------------|-----------------|
| Gregor Rot           | 100        | 784   | yes                         | yes             |
| Jernej Ule           | 100        | 1,847 | yes                         | yes             |
| Bin Zhang            | 100        | 536   | yes                         | yes             |
| Christian von Mering | 100        | 4,950 | yes                         | yes             |
| Nenad Ban            | 100        | 575   | yes (standalone comparison) | yes             |

Nenad Ban could initially not be verified through the server function because of a pre-existing bug, see section 6; the standalone algorithm comparison matched, and after the fix the server function returns the same 100 nodes and 575 edges.

## 6. Other changes made in the same session

- **ORCIDs ending in X.** `get_author_network` lowercases the search string, but the authors table stores the ORCID check digit as an uppercase X. A lookup for 0000-0002-9527-210x therefore found no row and about 9% of all authors (those whose ORCID ends in X) always produced an empty network. The ORCID token is now uppercased before the lookup (`index.py`, one line).
- **dbase/run.sh** contained unresolved git merge-conflict markers and could not run; resolved.
- **dbase/README.md** added: when to rebuild versus restart, the commands, and smoke tests.
- **authors\_all column dropped.** It was only ever written by `openalex/db_publications.py`; the server never selects it and the frontend only receives what the server returns. The build script no longer stores it. The existing database was rewritten without the column into `openalex/pubscan_slim.db` (28,682,355

publications, 5,117,880 authors, integrity check ok): 13.69 GB to 9.38 GB. Server output from the slim file is byte-identical to the full file for all test authors. Deploying it means copying it over db/pubscan.db and restarting the container; this was left for the operator.

## 7. Files touched

| File                        | Change                                                         |
|-----------------------------|----------------------------------------------------------------|
| pubscan/index.py            | Change A in get_author_network; ORCID uppercased before lookup |
| dbase/Dockerfile            | Change C: background cat of db/*.db before Apache starts       |
| dbase/run.sh                | merge-conflict markers resolved                                |
| dbase/README.md             | new: rebuild and restart instructions                          |
| openalex/db_publications.py | authors_all column no longer created or stored                 |
| openalex/pubscan_slim.db    | new, untracked: current database without authors_all (9.38 GB) |

Measurement setup: 8 cores, 62 GB RAM, /dev/md2 RAID0 of two 3.6 TB HDDs, Python 3.12, SQLite 3.45. Warm numbers are the second of two consecutive runs; cold numbers are the first request after the pages had been evicted.